

CardStack

Design D1: Detailed Design

Goal Statement

CardStack is a payment system that allows consumers to complete online purchases quickly while automatically selecting the user's credit card that provides the best rewards for the purchase category.

1. Scope

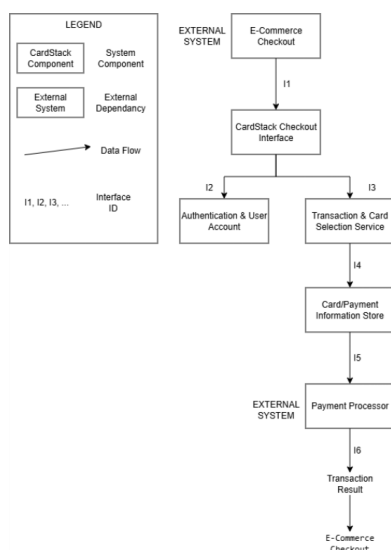
This document details the high level structure for the use case of a user checking out with CardStack feature on a third party ecommerce website and utilizing the mobile app to add authorized cards.

D1 Implementation

This documentation will outline the workflow for the checkout, API contracts between the frontend component and the backend server, and the SQL structures needed.

D2 Deferred

The structure for the mobile and web apps will be deferred to D2 design documentation



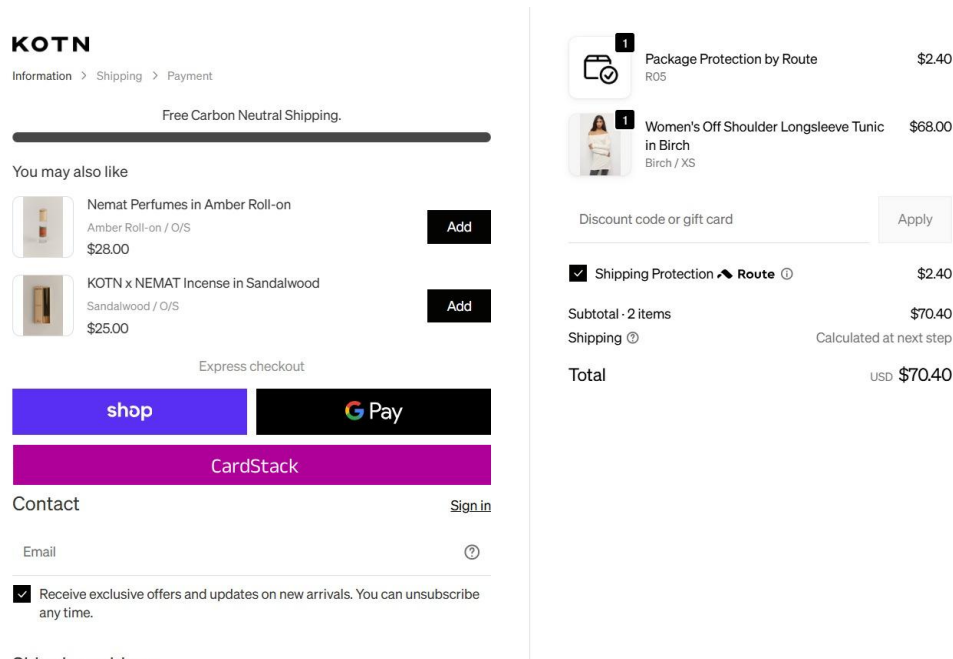
- **Rounded rectangles** represent CardStack components developed by the team.

- **Gray/external boxes** represent external systems that CardStack depends on.
- **Solid arrows** represent data or control being sent between components.
- **Interface IDs (I1, I2, etc.)** identify the interfaces described in the Interface Specification Table.
- **Directional arrows** indicate the direction of information flow.

2. Data Model and Diagram

Checkout Button

The checkout button will be attached to a third-party ecommerce site as an option for a user to pay. This component is a frontend feature that is visible to the user.



This is an entity because merchants will need to integrate our application into their stores through this button that will then launch login and checkout. This is driving the core operation of the service.

One merchant can have many checkout buttons, but each button will have only one corresponding merchant.

Indexed: button id, allowed domain

Button id will be looked up on every page load. Must be fast and unique. Allowed domain ensures the functionality only works on authorized sites.

We will use a relational model for safety and security.

Other components:

Login popup + checkout pages

Automatic card selection

Sends webhook to merchant with payment result

Adding transaction to user history

Receipt forwarding

IOS/Android Mobile App

The mobile app will be an area for users to configure account settings, add credit card and bank account information, and view points earned and what card was used for any transaction. Creating an app lets users easily see their information on the go.

Web Application

The website will be an area for users to configure account settings, add credit card and bank account information, and view points earned and what card was used for any transaction. The website, in future designs, will let users view more features and settings that the mobile app will not contain, due to restraint.

Personal CardStack Debit Card

For the second phase of CardStack, which for now is in concept testing to see if its possible, will allow users to obtain a virtual debit card to run their transactions through. The concept of the virtual debit card is to still use the CardStack algorithms for in person transactions.

3. Core Algorithms Needed

Card Processing

Problem it solves

Once a transaction request with the user's specific card is chosen, the card will need to be processed and charged.

Inputs and outputs

The userId, transaction details, connected credit card Id, and card information will be inputted. A success/error message, the userId, and the connected credit card Id will be outputted.

Expected Complexity

The why

Since we are the middle man trying to determine which card to use, we are also responsible for running the transaction.

Edge Cases

On fail, the algorithm will output an errorId explaining what exactly happened. From there, another algorithm will handle the appropriate response to send back to the third party ecommerce site.

Financial Institution Setup

Problem it solves

Lets users connect their credit cards to their CardStack account for the algorithm to use.

Inputs and outputs

User cards and accounts, user information, preferred cards will be inputted. A selected card will be outputted for every transaction.

Expected Complexity

Will be technically, legally, and operationally complex

The why

This is the basis for the app functionality. By allowing users to upload multiple cards, we can then automatically select the appropriate card per transaction.

Edge Cases

Users upload only one card. Users upload multiple cards with same reward percentages per category. Users preferred card is at its limit. Comparing cards that earn miles with cash back cards.

4. Build vs Reuse

	Build or Reuse	Library/Service	License	Reason
payment processor	reuse	Payment nerds	NA	Reuse for speed and safety
External login service	reuse	Google API	google	Common service that already has trust
Bank connecting	reuse	plaid	plaid	Security risk
Per card information database	build	supabase	NA	Unique information to tool built by team research for accuracy
Card selection algorithm	build	golang	NA	Unique to tool

5. API Contracts

All variables in the api contract inputs and outputs are required except for the error. The error field will contain a custom backend error code made from the server. From that code the frontend will determine the next actions. Error codes for these json contracts are HTTP 500. Versioning of the below json contracts are “/server/v1”. There are no specific groups.

Card Processing

Endpoint name: /process-card-payment

Input:

```
{
  cardId: "fewf-we43-efw2-dwe1" (Must be UUID)
  userId: "cardstack-user-ed42" (Must start with "cardstack-user-")
}
```

```
transactionDetails: {  
    transactionId: "ej4o-3kdi-dfkr-do3s" (String - Must be UUID)  
    amount: "132.34" (Float64 - Must be integer of any range)  
    category: "merchandise" (String - No boundary)  
    thirdPartyInstitution: "amazon" (String - No boundary)  
}  
}
```

Output:

```
{  
    Success: "true" (Bool - Must be true or false)  
    Error: "nil" (String - No boundary but must be an error code recognized by the  
frontend)  
    transactionId: "ej4o-3kdi-dfkr-do3s" (String - Must be UUID)  
}
```

Error Response:

```
{  
    Success: "false" (Bool - Must be true or false)  
    Error: "CardProcessTransactionError" (String - No boundary but must be an error  
code recognized by the frontend)  
    transactionId: "ej4o-3kdi-dfkr-do3s" (String - Must be UUID)  
}
```

Financial Institution Setup

This endpoint will be called from Plaid's webhook once an institution attempt happened

Endpoint name: /connect-bank-account

Input:

```
{  
    userId: "cardstack-user-ed42" (String - Must be UUID)
```

```
institutionId: "ejr4-dire-qowe-3wi4" (String - Must be UUID)
accountId: "doer-for4-djeo-ek45" (String - Must be UUID)
success: "true" (Bool - Must be true or false)
}
```

Output:

```
{
  RedirectUrl: "cardstack.com/connected-accounts" (String - Valid url)
  Success: "true" (Bool - Must be true or false)
  Error: "nil" (String - No boundary but must be an error code recognized by the
frontend)
}
```

Error Response:

```
{
  Success: "false" (Bool - Must be true or false)
  Error: "AddInstitutionError" (String - No boundary but must be an error code
recognized by the frontend)
  institutionId: "ej4o-3kdi-dfkr-do3s" (String - Must be UUID)
}
```

6. Technology Choices

Frontend

The frontend will be built using React Vite. Vite allows cross platform, reusable components, for web and mobile applications with little change. As CardStack looks to expand creating a mobile app is a priority.

Backend

The backend will be used with the language Golang (GO). Go is powerful for creating an api server as it allows thousands of process to run using minimal memory. As we look to use Plaid too, GO is their primary language and has high support for the functions.

Database

SupaBase is a good choice for our database service because it's a good fit for the team as we have experiencing using the platform, its free and does not require any licensing.

Supabase is highly regarded by many developers for its ease of use and quality. Because many developers are using supabase, there is a lot of community support.

Servers

Google Cloud will be used to host our backend servers. For our startup Google Cloud offers an incredible free tier program that allows millions of requests, 180,000 vCPU-seconds, and 360,000 GiB-seconds.

Hosting

Firebase will be used to run the frontend hosting. Firebase is a part of the Google ecosystem so pairing with the backend servers will be easy. Firebase also offers custom domains to use so we can avoid going to another party, line NameCheap, to connect a custom domain.